

2025春编译原理 - cyk

材料说明

- 本文：学习方法和试题回忆
- lab：能通过zqf老师当堂给的测试用例
- lec：2025年zqf老师的课件
- 题册和答案：见下文
- 期末题：网上找的期末题：建议课刷完，题册写完有时间再做
- 课程复习资料：网上找的笔记和选择题答案等
- 24春_编译原理：cyh学长的火炬
- 编译原理郭庆吉：gqj学姐的火炬
- 特别感谢lfd学长、cyh学长和gqj学姐

笔者成绩：86.8

详细信息			
任务号：	2024-2025-2-22CS22014-008	学年学期：	2025春季
课程代码：	22CS22014	课程名称：	编译原理
累加分项计分制：	百分制	总评计分制：	百分制
计分制满分：	100.0	总成绩：	86.8
平时考核：	权重（占总成绩百分比） [10.0%]，满分[10.0]	本项得分：	9.9（折算分数：9.9）
实验：	权重（占总成绩百分比） [20.0%]，满分[100.0]	本项得分：	90.3（折算分数：18.06）
期末：	权重（占总成绩百分比） [70.0%]，满分[100.0]	本项得分：	84.0（折算分数：58.8）

取消

教学班排名：8

学习方法：

- 刷mooc，关注课件中的例题
- 做题册，答案：[答案1](#) [答案2](#)
 - 题册中带星号的不用管

- 类似这种非标准的sdt也不用耽误太多时间

➤ Pascal语言的标准for语句

for v := E1 to E2 do S1

定义成和下面的代码序列有同样的含义：

begin

v := E1;

while v <= E2 do begin

S1;

v := succ(v);

end

end

请为for语句设计一种合理的中间代码结构，并写出产生中间代码的翻译方案

实验

难度大，代码量很大，自己完成不太现实。如果改的不多的话直接用我的吧，查重和验收都不是很严。

考试回忆：

题量超级大，我最后一题只答了不到一半，下面是能回忆起来的题

选择

就记得一道，如果一个文法是（）文法，他就一定是LR(1)文法 A、LR(0) B、SLR(1) C、LALR(1) D、以上全不对
当时没说是单选，这四个文法是包含关系，我写的ABC，不知道对错，因为比较怪所以记下来了 其余跟spoc上选择题关系密切，把spoc习题做一下再考试

填空（顺序忘了，可能漏了一题）

1. 下图，给全文，把过程和活动记录挖空

- 使用**过程**(或**函数**、**方法**)作为用户自定义动作的单元的语言，其编译器通常以**过程**为单位分配存储空间
- 过程体的每次执行称为该过程的一个**活动**(*activation*)
- 过程每执行一次，就为它分配一块连续存储区，用来管理过程一次执行所需的信息，这块连续存储区称为**活动记录**(*activation record*)

2. 下图，类似全文，填堆和栈

变长数据的存储分配

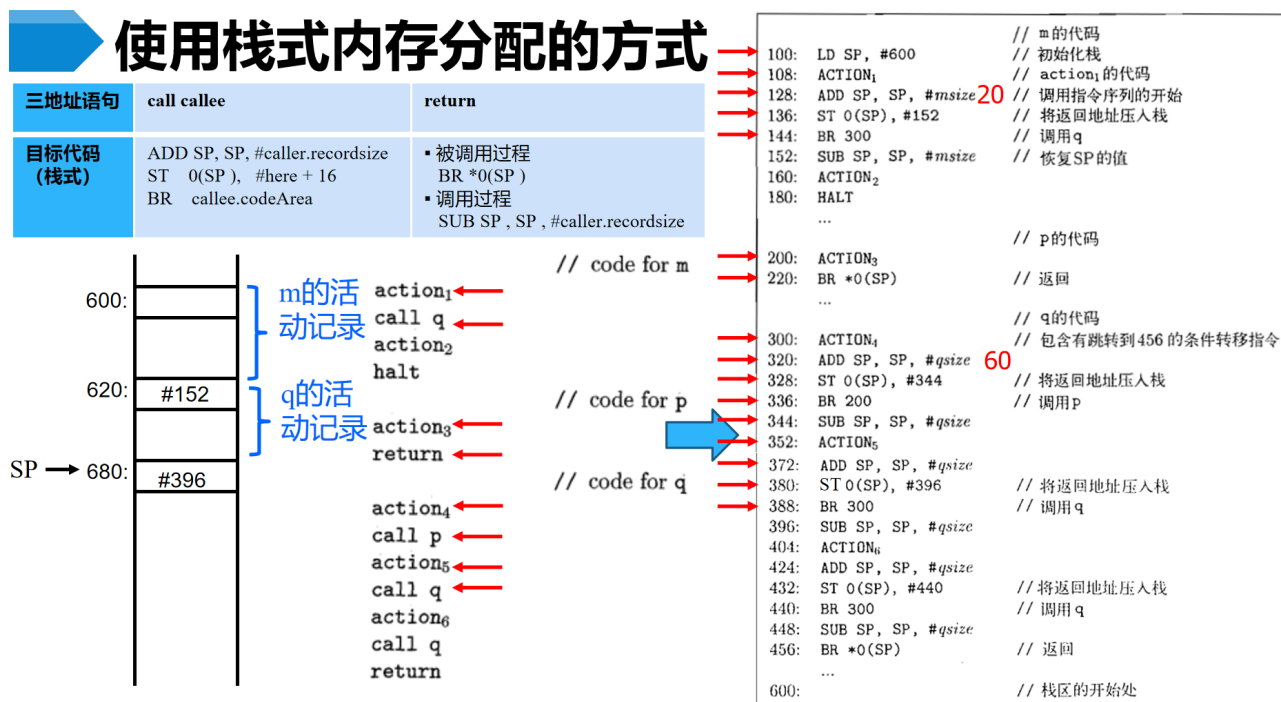
- 在**现代程序设计语言**中，在**编译时刻不能确定大小的对象**将被分配在**堆区**。但是，如果它们是**过程的局部对象**，也可以将它们分配在**运行时刻栈**中。尽量将对象放置在栈区的原因：可以**避免**对它们的空间进行**垃圾回收**，也就**减少**了相应的**开销**
- 只有一个数据对象**局部于某个过程**，且**当此过程结束时它变得不可访问**，才可以使用**栈**为这个对象分配空间

3. 下图，问寄存器描述符和地址描述符定义

- **寄存器描述符** (*register descriptor*)
 - 记录每个**寄存器**当前存放的是哪些变量的值
- **地址描述符** (*address descriptor*)
 - 记录运行时每个**名字**的当前值存放在哪个或哪些位置
 - 该位置可能是寄存器、栈单元、内存地址或者是它们的某个集合
 - 这些信息可以存放在该变量名对应的**符号表**条目中

4. 原理类似下图，给出假设：指令中每一个操作数或一个操作码占四字节（言外之意其余0(SP)这种不占），给了m和q活动记录的长度、二者代码区的开始位置，还给了表格中call的三条栈式目标代码，让你把里

面的标签替换成具体的绝对地址（类似右图这种立即数的形式）。



大题

1. 词法分析

- 设计token序列（原理类似下图）

```

float limitedSquare(x) {float x ;

/* returns x-squared, but never more than 100 */

return (x<=-10.01 || x>=10.0)?100:x*x;

}

```

➤ 解:

- | | |
|--------------------------------------|-----------------------------|
| 1、float <FLOAT, -> | 16、x <IDN, x> |
| 2、limitedSquare <IDN, limitedSquare> | 17、>= <COMPARISON, >=> |
| 3、(<SLP, -> | 18、10.0 <CONST_FLOAT, 10.0> |
| 4、x <IDN, x> | 19、) <SRP, -> |
| 5、) <SRP, -> | 20、? <CONP, ?> |
| 6、{ <LP, -> | 21、100 <CONST_INT, 100> |
| 7、float <FLOAT, -> | 22、: <CONP, :> |
| 8、x <IDN, x> | 23、x <IDN, x> |
| 9、; <SEMI, -> | 24、* <ARITHMETIC, *> |
| 10、return <RETURN, -> | 25、x <IDN, x> |
| 11、(<SLP, -> | 26、; <SEMI, -> |
| 12、x <IDN, x> | 27、} <RP, -> |
| 13、<= <COMPARISON, <=> | |
| 14、-10.01 <CONST_FLOAT, -10.01> | |
| 15、 <LOGICAL, > | |

将14行分成两行，负数分开为两部分，负号和常量

- 写单词的正则表达式，画它的DFA（以字母或下划线开头，然后是若干字母或数字）但是没弄清楚“若干”包不包括零个，我写的：

```
letter -> a|b|...|z|A|B|...|Z
digit -> 0|1|...|9
word -> (_|letter)(letter|digit)*
```

自动机就很好画了。

2. 语法分析

- 很常规，给一个文法，证明是LR(1)文法但不是LALR(1)文法 增广文法，画LR(1)自动机，找同心项集，举例说明合并后有规约-规约冲突即可
- 还要证明改造后的文法不是LL(1)文法 提左公因子，消除左递归，FIRST集，FOLLOW集，SELECT集，说明同一左部的产生式的SELECT集有相交

3. 语法制导翻译

原题（强调了是S-SDD）

$$S \rightarrow (L) \mid a$$

$$L \rightarrow L, S \mid S$$

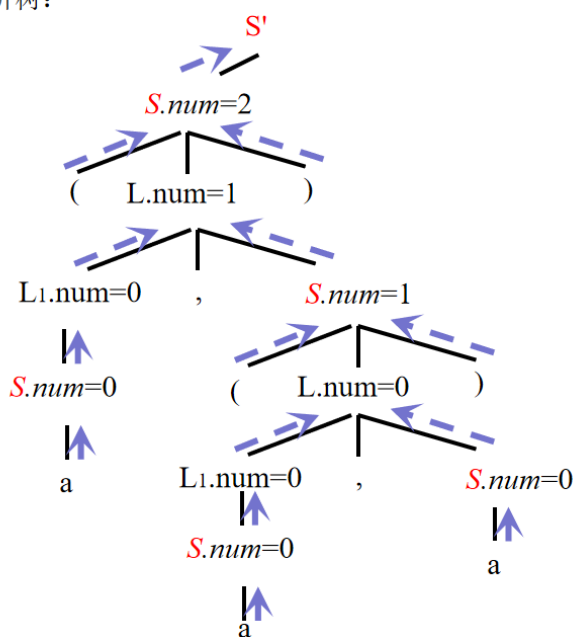
➤ (1) 试设计一个SDD，输出S所生成的串中的配对括号的个数。

举例：句子是(a, (a, a))

输出结果为2。

注释分析树：

产生式	语义规则
(1) $S \rightarrow S'$	$\text{print}(S.\text{num})$
(2) $S \rightarrow (L)$	$S.\text{num} = L.\text{num} + 1$
(3) $S \rightarrow a$	$S.\text{num} = 0$
(4) $L \rightarrow L_1, S$	$L.\text{num} = L_1.\text{num} + S.\text{num}$
(5) $L \rightarrow S$	$L.\text{num} = S.\text{num}$



还要写成栈的操作，注意是规约时执行语义动作就好了 注意：文法需要增广，上图(1)应该是 $S' \rightarrow S$ ，这是为了输出根节点S的配对括号数，还是文字游戏，“配对括号数”一左一右算一个还是两个，我写的两个，估计都算对

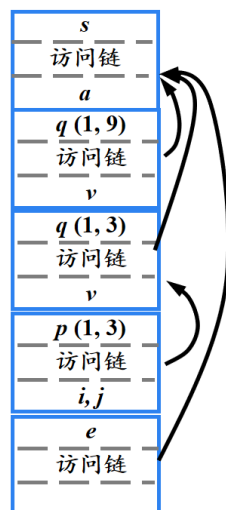
4. 运行存储分配

- 画控制栈，画访问链和控制链

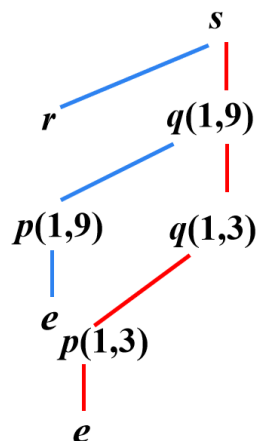


过程	嵌套深度
<i>sort</i>	1
<i>readarray</i>	2
<i>exchange</i>	2
<i>quicksort</i>	2
<i>partition</i>	3

控制栈



活动树

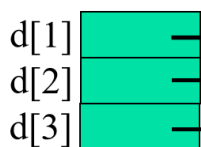


在上图的基础上每个活动记录的访问链上面加一条控制链，指向其调用者（上方紧邻）的局部数据开始处

- 相同的情形，画display表

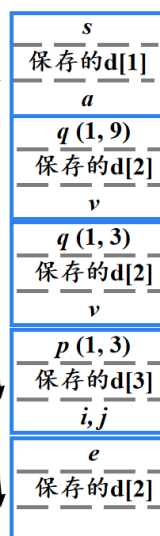


display表

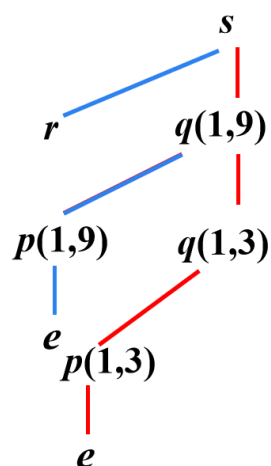


过程	嵌套深度
<i>sort</i>	1
<i>readarray</i>	2
<i>exchange</i>	2
<i>quicksort</i>	2
<i>partition</i>	3

控制栈



活动树



5. 中间代码生成

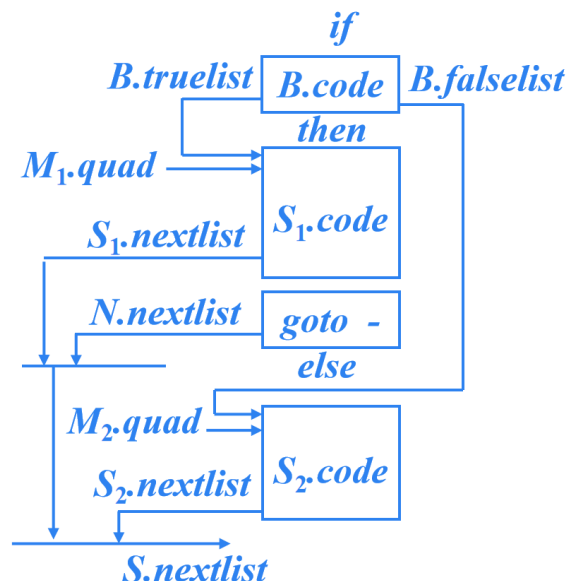
- 回填的SDT，给你产生式和框图，让你填几条代码（应该是这个，要不就是 $S \rightarrow \text{while } B \text{ do } S_1$ ）

$S \rightarrow \text{if } B \text{ then } S_1 \text{ else } S_2$

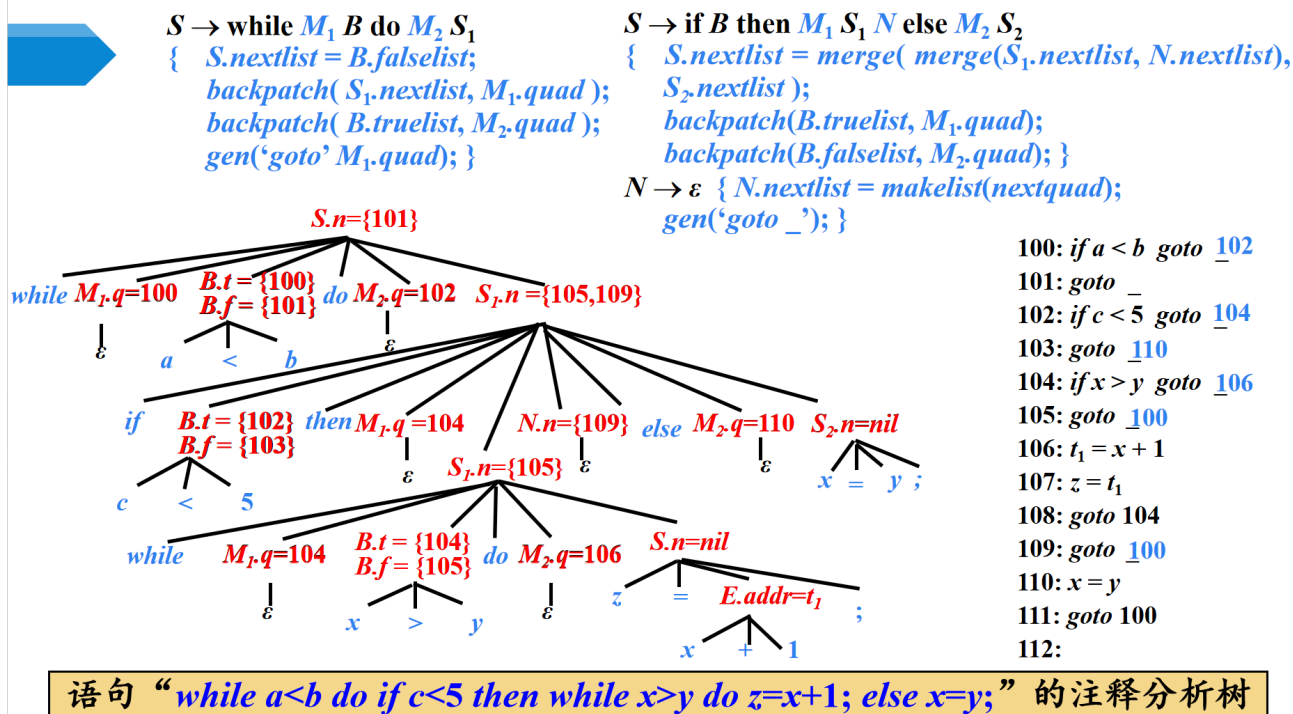
```

S → if B then M1 S1 N else M2 S2
{
    S.nextlist = merge( merge(S1.nextlist,
    N.nextlist), S2.nextlist );
    backpatch(B.truelist, M1.quad);
    backpatch(B.falselist, M2.quad);
}
N → ε
{
    N.nextlist = makelist(nextquad);
    gen('goto _');
}

```



- 给语句，给注释分析树，已知开始的语句编号100，让你填所有的list和quad，注意：没给上面的SDT



策略是草纸上硬写中间代码，然后根据各种list的定义（跳转到某处的语句集合）填空。

6. 代码优化

- 给中间代码，划分基本块，画流程图
- 根据流程图写出自然循环
- 写出所有基本块的def和use集合，写出各基本块出口处的活跃变量信息
- （记不清了，毕竟没写到这里）